VISION ARTIFICIELLE

# TP n°1

# Introduction au traitement d’images

| | |
|---|---|
| **Établissement** | EMSI Casa |
| **Étudiant** | Anwar Ezzirani |
| **Groupe** | 4 |
| **Année universitaire** | 2026–2027 |
| **Image étudiée** | Lenna_(test_image).png |

Compte rendu réalisé avec Python, NumPy et Matplotlib.

# 1. Objectifs du TP

Ce TP a pour objectif de comprendre la représentation matricielle d'une image numérique et de programmer plusieurs traitements élémentaires : symétrie, négatif, conversion en niveaux de gris, extraction des canaux RGB, calcul d'un histogramme et égalisation de l'histogramme afin d'améliorer le contraste.

# 2. Chargement et caractéristiques de l'image

L'image fournie est une image couleur RGB de dimensions 512 × 512 × 3. Après chargement, les composantes sont converties au type uint8, donc chaque valeur est comprise entre 0 et 255. L'origine des coordonnées est située en haut à gauche.

```
image = charger_image("Lenna_(test_image).png")
print(image.shape)  # (512, 512, 3)
print(image.dtype)  # uint8
```

# 3. Fonctions élémentaires

## Question 1 — Symétrie verticale

Pour une image de largeur p, le pixel de coordonnées (i, j) reçoit la valeur du pixel (i, p − 1 − j). L'ordre des colonnes est ainsi inversé.

```
def symetrie(im):
    n, p, c = im.shape
    im_sym = np.zeros_like(im)
    for i in range(n):
        for j in range(p):
            im_sym[i, j] = im[i, p - 1 - j]
    return im_sym
```

## Question 2 — Négatif

Chaque composante de couleur v est remplacée par son complément dans [0, 255], soit 255 − v.

```
def negatif(im):
    n, p, c = im.shape
    im_neg = np.zeros_like(im)
    for i in range(n):
        for j in range(p):
            im_neg[i, j] = 255 - im[i, j]
    return im_neg
```

## Question 4 — Conversion en niveaux de gris

La luminance est calculée avec la relation y = 0,2126 r + 0,7152 g + 0,0722 b, puis arrondie à l'entier le plus proche. Le poids du vert est le plus important, car l'œil humain est plus sensible à cette composante.

```
def niveaudegris(im):
    n, p, c = im.shape
    im_lum = np.zeros((n, p), dtype=np.uint8)
    for i in range(n):
        for j in range(p):
            r, g, b = im[i, j].astype(float)
            im_lum[i, j] = np.rint(
                0.2126*r + 0.7152*g + 0.0722*b)
    return im_lum
```

## Questions 5, 6 et 7 — Canaux RGB

Une matrice nulle de même dimension est créée. Seul le canal souhaité est ensuite recopié : indice 0 pour le rouge, 1 pour le vert et 2 pour le bleu.

```
def image_rouge(im):
    resultat = np.zeros_like(im)
    resultat[:, :, 0] = im[:, :, 0]
    return resultat

# Même principe avec l'indice 1 pour le vert et 2 pour le bleu.
```

# 4. Résultats des transformations

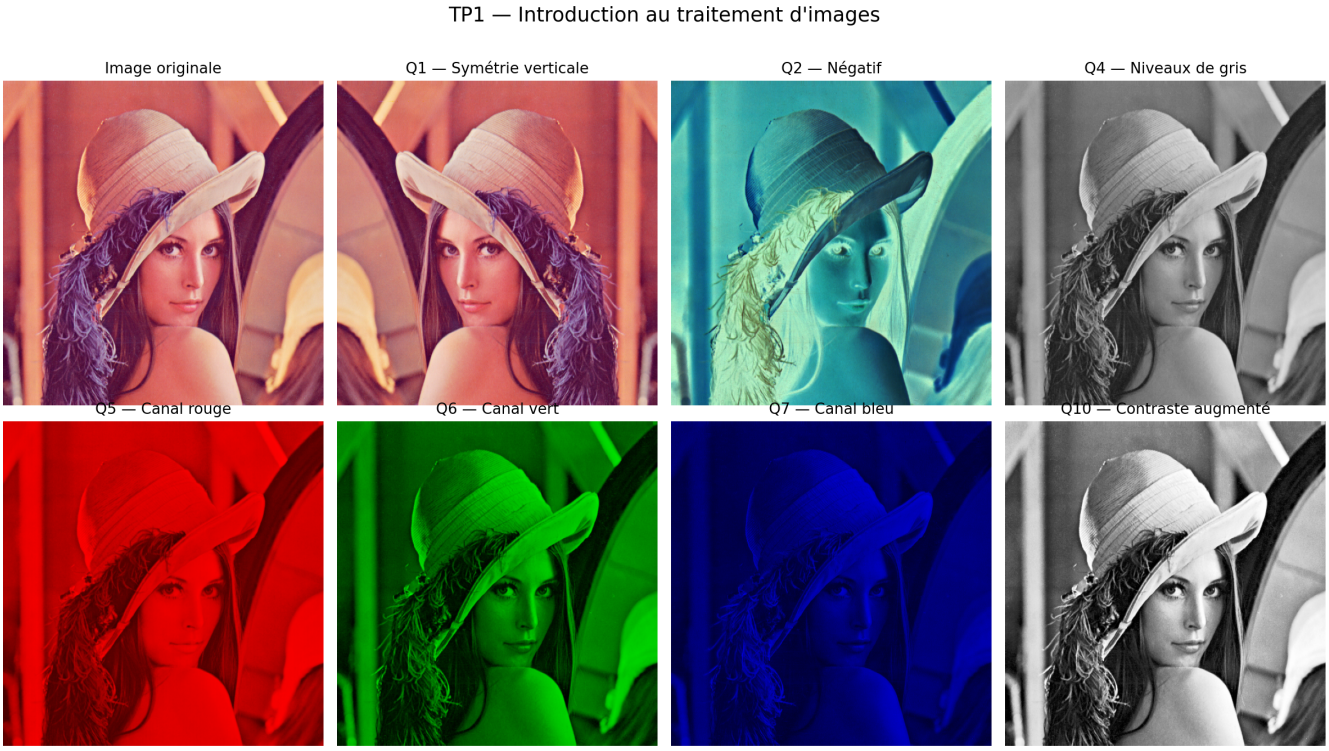



Figure 1 — Image originale, symétrie, négatif, luminance, canaux RGB et image après égalisation.

# 5. Histogramme et amélioration du contraste

## Question 8 — Histogramme

L'histogramme possède 256 cases. Chaque case H[k] indique le nombre de pixels de niveau de gris k. La somme de l'histogramme est égale à 512 × 512 = 262 144 pixels.

```
def histo(im):
    H = np.zeros(256, dtype=np.int64)
    for i in range(im.shape[0]):
        for j in range(im.shape[1]):
            H[int(im[i, j])] += 1
    return H
```

## Question 9 — Égalisation de l'histogramme

L'histogramme cumulé hc est normalisé dans [0, 255]. Le tableau obtenu constitue une table de conversion monotone associant à chaque intensité une nouvelle intensité.

```
def egaliser_histogramme(H, gray_image):
    hc = np.zeros(len(H))
    hc[0] = H[0]
    for i in range(1, len(H)):
        hc[i] = hc[i - 1] + H[i]
    he = (hc / gray_image.size) * 255
    return he
```

## Question 10 — Augmentation du contraste

Chaque pixel de l'image en niveaux de gris est remplacé par la valeur indiquée dans la table d'égalisation. Les niveaux sont alors répartis sur une plage plus large, ce qui améliore le contraste global de l'image.

```
def augmenter_contraste(gray_image):
```

```
H = histo(gray_image)
he = egaliser_histogramme(H, gray_image)
table = np.rint(he).clip(0, 255).astype(np.uint8)
new_image = gray_image.copy()
for i in range(gray_image.shape[0]):
    for j in range(gray_image.shape[1]):
        new_image[i, j] = table[gray_image[i, j]]
return new_image
```

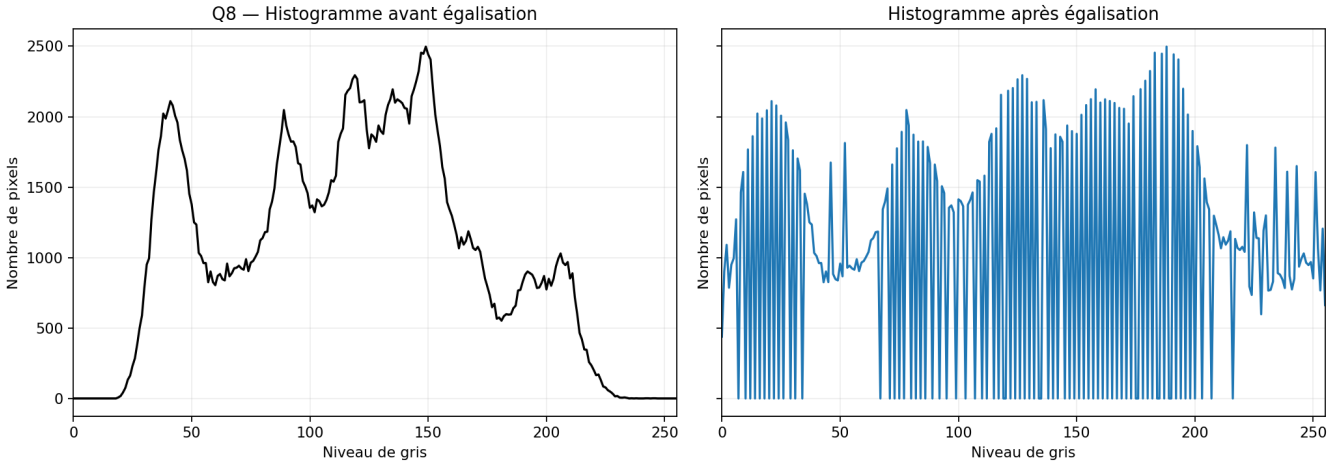



Figure 2 — Répartition des niveaux de gris avant et après égalisation.

# 6. Vérification

| Contrôle | Résultat |
|---|---|
| Dimensions de l'image | 512 × 512 × 3 |
| Type des pixels | uint8 |
| Somme de l'histogramme initial | 262 144 |
| Somme de l'histogramme égalisé | 262 144 |
| Tests fonctionnels | Réussis |

# 7. Conclusion

Ce TP a permis de manipuler une image comme un tableau NumPy et de comprendre le rôle des coordonnées, des composantes RGB et de la luminance. Les transformations demandées ont été réalisées et vérifiées. L'égalisation de l'histogramme répartit davantage les intensités disponibles et produit une image dont les détails sont globalement plus contrastés.